

SmartHire GenAI — Resume Matching & AI Career Mentor

Katakam Harsha | NIT Puducherry | GenAI Project
September 2026

1. What the Project Does

SmartHire GenAI is a career portal that I built to help with the job search process. The main flow is simple: the user uploads a resume, SmartHire reads it, finds related jobs, gives CV improvement suggestions, and provides career advice through an AI Career Mentor.

There are four main features. The **Resume Parser** reads a PDF, DOCX, or TXT resume, or alternatively one to four cropped screenshot images of a resume, and extracts details such as skills, education, experience, and target role into a structured profile. The screenshot path lets a candidate share only the non-identifying part of their resume when they are not comfortable uploading the full file. The **Semantic Job Search** compares the meaning of the candidate profile with job postings using embeddings and returns matching jobs.

The **CV Improvement** feature uses the resume and a target job to suggest missing skills, weak points, and better wording. The **AI Career Mentor** answers questions about jobs, skills, and career paths using the project's job postings and career notes through RAG.

2. Changes Made During Development

The project changed quite a bit during development. I first worked with a larger job dataset, but embedding and testing it was slow on my machine and also used more API requests. I therefore reduced and normalized the data for the final version.

The current job data used by the project has **80 job records across 20 job roles** (e.g. Software Engineer, Data Scientist, Data Analyst, Machine Learning Engineer, Python/Java/PHP/React Developer, Backend/Frontend/Full Stack Developer, Cloud Engineer, Data Engineer, GCP Data Engineer, Mobile App Developer, Business Analyst, Application Developer, Application Support Engineer, and Software Development Engineer). After changing the dataset, I rebuilt the job vector index so that the stored embeddings matched the new CSV. This made local testing faster and also made the deployment setup smaller.

I also moved the project towards a **Streamlit-only application** instead of keeping a separate React interface. The goal was to keep one simple application that can be tested locally and later deployed on Streamlit Community Cloud.

3. API and Model Testing

I tested different API options during development instead of choosing the first provider and building the whole project around it. The providers I checked included **OpenRouter, Token Router, Groq, and Gemini**. The main things I compared were model access, API behaviour, quotas, and whether the setup was practical for this project.

Some of the testing caused quota or temporary API problems, so the final setup had to handle those cases instead of assuming that an API would always be available.

In the final version, **Gemini is the main provider**. It is used for generation and, by default, for embeddings. I chose Gemini embeddings instead of loading a larger local neural embedding model because the planned Streamlit Community Cloud free tier has a roughly **1 GB RAM limit**. Keeping the embedding work through the API reduces the memory used by the deployed application.

The code still has local alternatives such as LSA and FastEmbed for testing, but these are not the default deployment path. The Gemini embedding code also uses batching and rate limiting to avoid sending too many embedding items too quickly. The generation calls also fall back to alternate Gemini models automatically if the primary model returns a temporary availability/high-demand error, so a single provider hiccup does not fail the whole request.

4. AI Career Mentor and RAG Pipeline

The Career Mentor was built as a real **five-step LangChain pipeline**, not just as one prompt sent directly to the model:

Guardrail → Retrieve → Prompt → LLM → Format Output

The guardrail checks the question before the LLM is called. If the question is outside the career, resume, job-search, or job-preparation scope, it is rejected.

For valid questions, the system retrieves relevant job postings and career notes, blending embedding similarity with keyword overlap, and only proceeds to generation if the best retrieved evidence clears a minimum relevance threshold; otherwise the mentor explicitly says it does not have enough evidence rather than guessing. The retrieved text is then placed into the mentor prompt before Gemini generates the answer. If the generated answer looks truncated, the pipeline automatically retries generation once before returning it. The final step formats the response for the Streamlit interface, including per-source relevance scores.

I also changed the mentor prompt during development. The earlier prompt mainly asked the model to be helpful. The final prompt tells it to use retrieved evidence first, avoid invented facts, separate career guidance from actual job-posting requirements, and show source labels such as [S1] and [S2]. This makes the answers easier to check against the project data.

5. Guardrails and Safety

Guardrails are used before the relevant LLM calls. The mentor is meant to answer career, resume, job-search, and job-preparation questions. Questions outside that scope are rejected, and the same guardrail also blocks prompt-injection style attempts (e.g. instructions asking the model to ignore its rules or reveal its system prompt).

I tested three deliberately out-of-scope cases: a medical diagnosis request, a request to create password-stealing malware, and a political-party recommendation. All three were refused, so the refusal test passed with **3/3 or 100% accuracy**.

6. Evaluation Results

I used a small starter benchmark. It is useful for checking the current system, but it is not a large statistical evaluation. The reported results were:

Metric	Result
Retrieval Top-1 hit rate	100%
Retrieval Top-5 hit rate	100%
Retrieval Top-10 hit rate	100%
Mentor correctness	5.0 / 5
Mentor grounding	5.0 / 5
Mentor helpfulness	5.0 / 5
Out-of-scope refusal accuracy	100%

All **12** retrieval queries found an expected job category (spanning roles such as Software Engineer, Data Scientist, Data Analyst, Machine Learning Engineer, PHP Developer, React JS Developer, Mobile App Developer, Cloud Engineer, Frontend Developer, and Application Support Engineer) in Top-1, Top-5, and Top-10. **8** in-scope mentor questions received 5/5 for correctness, grounding, and helpfulness. These included Data Scientist, Software Engineer, Data Analyst, Machine Learning Engineer, Python backend, and NLP-focused Data Scientist questions, plus two harder comparison-style questions.

The two harder questions were: one comparing Data Scientist and Data Engineer roles using the included postings, and one asking whether someone must become a Senior Data Scientist before moving into ML engineering. Both were scored by the judge and received 5/5 on all three metrics, with the answers correctly grounded in the retrieved documents and making the required role distinctions.

The evaluation run used all **8** available Gemini judge calls, and all 8 in-scope mentor cases were scored – none were skipped. The judge budget is configurable and currently set to match the number of in-scope cases exactly, so a future run with more test cases would need a higher budget (or multiple runs) to score every case.

7. Prompt Before and After

I also compared the older mentor prompt with the final one. The older version mainly asked the model to answer clearly and be helpful. The new version added stronger rules about using retrieved evidence, not inventing information, handling career-path questions, and adding source labels.

The comparison was completed in the final evaluation run. Earlier runs sometimes skipped this test because Gemini was temporarily unavailable or a quota was reached.

8. What Worked Well

The semantic job search worked well on the current test set, with 100% hit rates at Top-1, Top-5, and Top-10 across all 12 retrieval queries. The mentor also gave grounded, top-scoring answers for all 8 judged cases, including the two harder comparison questions.

Another thing that worked well was keeping the application light enough for the planned deployment. Reducing the dataset to 80 rows and using Gemini embeddings avoided the need to load a large local embedding model.

The API testing was also useful because it showed that provider choice, quotas, and temporary failures can affect a GenAI application. This led to a more careful final setup, including automatic fallback models and retry handling for temporary Gemini errors.

9. What I Would Improve

With more time, I would first use a much larger job dataset. I would also create a larger evaluation set with more difficult queries, especially questions that combine several job roles or use incomplete resume information, and raise the judge-call budget to match.

I would also evaluate the CV improvement feature more directly. For example, I would compare the resume-to-job match before and after applying the suggested changes.

Finally, I would do more testing on the deployed application, including repeated requests, API limits, response time, and cold starts.

10. Known Limitations and Conclusion

The main limitation is the current dataset size: **80 jobs across 20 roles**. This is enough to demonstrate the project, but it is much smaller than a real recruitment dataset. Therefore, the 100% retrieval result should only be understood as the result on the current test set, not as proof of the same performance on a much larger dataset.

The Career Mentor is also limited by the information in its knowledge base. If the required information is not in the retrieved job postings or career notes, the mentor may not have enough evidence to give a reliable answer, and it is designed to say so rather than guess.

The project also depends on the Gemini API, so quotas, temporary API failures, latency, and model behaviour can affect the application. The evaluation is a small development benchmark and is run manually rather than as part of the deployed Streamlit application.

Overall, SmartHire GenAI now brings resume parsing, semantic job matching, CV suggestions, and a RAG-based Career Mentor into one Streamlit application. The current version works well for the test data, while the main areas for future work are a larger dataset, a larger evaluation set, and more deployment testing.